-Programa: conjunto independiente de instrucciones usadas para operar una computadora con el fin de producir un resultado específico
-Computadora: Máquina electrónica capaz de almacenar información, tratarla y resolver problemas mediante determinados programas
-Programar:

Lenguajes de programación:

a) Lenguaje de máquina (1° generacion): Codificacion en secuencias de dígitos binarios.
b) Lenguajes ensambladores (2° generacion): De bajo nivel, son códigos mnemotécnicos. Requieren de una traducción (ensamblador) para traducirlo a lenguaje de máquina
c) Lenguajes de alto nivel (3° generacion): Son amigables al programador.
    i) Cuando cada declaración es traducida individualmente y ejecutada inmediatamente, el lenguaje se llama INTERPRETADO y el programa que lo traduce INTÉRPRETE
    ii) Cuando todas las instrucciones son traducidas como una unidad completa antes que cualquier declaración sea ejecutada, el lenguaje se llama COMPILADO y el traductor COMPILADOR

Los programas escritos en un lenguaje de computadora (de alto o bajo nivel) se conocen como programas fuente y código fuente.

Orientaciones a procedimientos y a objetos:
En un lenguaje **orientado a procedimientos** las instrucciones disponibles se usan para crear unidades independientes, conocidas como procedimientos. El propósito de un procedimiento es aceptar datos como entrada y transformarlos de alguna manera para producir un resultado específico como una salida.

Usando un **enfoque orientado a objetos**, un programa debe definir primero los objetos que manipulará, incluyendo una descripción de las características generales de los objetos y unidades específicas para manipularlos, como cambiar el tamaño y la posición y transferir datos entre objetos.

Software de aplicación y de sistema:

El **software de aplicación** consiste en aquellos programas escritos para realizar tareas particulares requeridas por los usuarios.
El **software de sistema** es la colección de programas que deben estar disponibles en cualquier sistema de cómputo en el que ha de operar.
Por lo tanto, el conjunto de programas de sistema usados para operar y controlar una computadora se llama **sistema operativo**, el cual realiza las siguientes tareas: administración de memoria; asignación de tiempo de CPU; control de unidades de entrada y salida como teclado, pantalla e impresoras, y la administración de todos los dispositivos de almacenamiento secundarios. se conocen como sistemas **multiprogramados y multitareas.**

SOLUCIÓN DE PROBLEMAS Y DESARROLLO DE SOFTWARE

En las ciencias y la ingeniería el enfoque se conoce como el método científico, mientras en el análisis cuantitativo el enfoque se denomina enfoque de sistemas.

El método usado para entender el problema que se va a solucionar y para crear una solución de software efectiva y apropiada se llama **procedimiento de desarrollo de software.** Consiste en tres fases:

a) Diseño y desarrollo: planteamiento de un problema o con una solicitud específica de un programa, lo cual se conoce como requerimiento de programa.
    i) Analizar el problema (10%): que debe hacer el programa, que salidas debe producir y que entradas se requieren.
    ii) Desarrollar una solucion (20%): seleccionar el conjunto exacto de pasos (algoritmo)
    iii) Codificar la solucion (20%): Estas estructuras controlan la forma en que el programa se ejecuta y consiste en los siguientes tipos:
        1) Secuencia: define el orden en que son ejecutadas las instrucciones
        2) Selección: proporciona la capacidad para hacer una eleccion entre diferentes operaciones, dependiendo del resultado de alguna condicion.
        3) Iteración (bucle): proporciona la capacidad para que la misma operacion se repita con base en el valor de una condicion.
        4) Invocación: implica invocar, o solicitar, un conjunto de instrucciones cuando sea necesario.
    iv) Probar y corregir (50%): verificar que el programa funciona en forma correcta y cumpla con sus requerimientos.
b) Documentacion: existen 5 documentos para toda solución de problema:
    i) Descripcion del programa
    ii) Desarrollo y cambios del algoritmo
    iii) Listado del programa bien comentado
    iv) Muestras de las pruebas efectuadas
    v) Manual del usuario
c) Mantenimiento: tiene que ver con la correccion continua de problemas, revisiones y la adicion de caracteristicas nuevas. Es la fuente principal de ingresos y la fase mas duradera.
d) Respaldo

Eficaz:

Eficiencia:

ALGORITMOS: Secuencia paso a paso de instrucciones que deben realizarse y describe como han de procesarse los datos para producir las salidas deseadas. Debe ser eficiente, preciso, definido y legible.

Puede escribirse o describirse en varias formas. Cuando se utilizan enunciados coloquiales para describirlo, se llama **pseudocódigo**. Cuando se usan ecuaciones matematicas, la descripcion se llama **formula.**

ALMACENAMIENTO:

Todas las computadoras deben realizar un conjunto minimo de funciones y proporcionar la capacidad para:

1) Aceptar entradas
2) Mostrar salidas
3) Almacenar informacion en un formato logico consistente
4) Ejecutar operaciones aritmeticas y logicas
5) Supervisar, controlar y dirigir la operacion y secuencia general del sistema

-Unidad de arimtetica y logica: Ejecuta todas las funciones de ese tipo
-Memoria central: Almacena informacion en un formato logico consistente (celdas), las cuales cada una tiene una direccion unica.
-Unidad de control: Dirige y supervisa la operacion general de la computadora.
-Unidad de entrada y salida: proporciona la interfaz a la que se conectan los perifericos.
-Almacenamiento secundario: Debido a que la memoria principal en grandes cantidades es relativamente cara y volátil (la info se pierde cuando no tiene energia), no es practica como area de almacenamiento permanente. Por esto, se usan dispositivos auxiliares o secundarios.

Memoria RAM: Memoria volátil, de acceso aleatorio, se puede leer y escribir en cualquier posicion

Memoria ROM: Memoria de solo lectura

-Los componentes físicos usados en la fabricación de una computadora requieren que los números y letras dentro de su unidad de memoria no se almacenen usando los mismos símbolos que la gente usa.
-La pieza más **pequeña y basica** de datos en una comp se llama BIT. (0 o 1)
-El **agrupamiento de ocho** bits para formar una unidad más grande se conocen como **bytes**.
-La colección de patrones consistentes en 0 y 1 que se usan para representar letras, dígitos individuales y otros caracteres individuales se llama **código de caracteres.**
-Los patrones usados para almacenar números se llaman **código de números.** El más comun para almacenar valores enteros dentro de una pc es la representacion de Complemento a dos (binario). cada posición de bit tiene un peso, o valor, de dos elevado a una potencia y un signo.
-Uno o más bytes pueden agruparse en unidades más grandes, llamadas **palabras**, lo cual facilita un acceso más rápido y más extenso a los datos. El número de bytes en una palabra determina los valores máximo y mínimo que pueden ser representados por la palabra.

Lenguaje C++: Lenguaje de procedimientos estructurados-

Librería: Conjunto de funciones predefinidas-

Cómo escribir un programa ejecutable:

1) Editor: programa que permite escribir y almacenar en disco un archivo de programa (bloc de notas)

2) Programa fuente: Puede comprender múltiples funciones (conjunto de archivos o funciones) que componen la estructura
3) Compilador: Comprueba si lo que se escribió es consistente. Traduce de lenguaje de alto nivel a código objeto o programa objeto (lenguaje de máquina). Ante errores no compila
    a) ARCHIVO .O permite compartir el programa sin mostrar como funciona.
4) Ligador: Combina el programa objeto con las bibliotecas.
5) Se obtiene finalmente, un programa ejecutable

ERRORES:

a) En tiempos de compilación: Errores de sintaxis
b) En tiempos de ejecución:
    i) Error lógico: errores en el diseño del logaritmo, solo son detectables en la prueba.
    ii) Error de ejecución: Cuando el programa ya fue compilado y cuando está en ejecución. (ej división por cero).

ESTRUCTURA DE UN PROGRAMA:

1) Directivas al preprocesador: procesa directivas, no son propias de c++
   ---> using namespace std; (le dice al compilador donde buscar para encontrar los archivos de encabezado en ausencia de cualquier designacion explicita adicional)
2) Declaraciones globales y compartidas
3) Función main ():

FUNCION MAIN

-Subprograma especial obligatorio y unico, que indica donde comienza la ejecucion del algoritmo.
-Bloque: Un bloque es una agrupación de instrucciones delimitadas por { y }. Unidad de ejecucion mayor que la secuencia y permite agrupar una secuencia de sentencias como una unidad.

-Se pueden colocar bloques dentros de otros

-Generan un ambito nuevo. (se puede definir una variable con el mismo identificador en diferentes bloques

Elementos útiles:

a) Variables: Almacenamiento de los datos en memoria
   Int a; -> Declaracion de una variable. El identificador no puede utilizar palabras reservadas, y debe ser de máx 32 caracteres.
b) Palabras reservadas: palabra apartada por el lenguaje para un propósito especial.
c) Delimitadores: indican comienzo o fin de una entidad
d) Comentarios: Texto marcado para ser identificado y obviado por el compilador. Facilita la legibilidad.

OPERADORES RELACIONALES: == (equivalente), != (diferente), <,>,<=,>=

Se escribe: Expresión1 Operador Expresión2

Las expresiones y el valor deben ser compatibles. Sirven para comprobar una condición, da siempre como respuesta un verdadero o falso.

OPERADORES LÓGICOS

! (not),
&& (and),

|| (or)
| (or bit a bit) Comparan bit a bit cada condición
& (and bit a bit)

-Funciones que solo pueden utilizarse con álgebra de bool.
-Se trabaja con evaluacion en cortocircuito: Si el de la izq es falso, no se evalua el de la derecha. (en caso del AND), para el OR es si fuera verdadero.
Ej: (n!=0) && (x<10/n) Si n =0 entonces n!=0 es falso, la 2da expresión no se evalua.
-Los resultados obtenidos en expresiones booleanas pueden almacenarse en una variable.

OTROS OPERADORES:
-Condicional ternario: ExpCond ? ExpV : Exp F; Devuelve el resultado cuyo valor depende de la condicion comprobada
-Asignación: variable = expresión
-Coma: Permite combinar 2 o mas expresiones separadas por coma en una misma linea.
-Especiales: ( ) para la llamada funcion, encerrando los argumenots y [] para dimensionar arreglos.
-Size of (mas abajo)

TIPOS DE DATOS: La elección de un determinado tipo u otro para una determinada entidad del programa proporciona info acerca de:
-El rango de posibles valores que puede tomar
-El conjunto de operaciones y manipulaciones aplicables
-El espacio de almacenamiento necesario para almacenar dichos valores
-La interpretación del valor almacenado
Tipos de datos **de clase**: tipo de dato creado por el programador

Tipos de datos **integrados**: es proporcionado como una parte integral del compilador

Tipos simples: sus valores son indivisibles

-int: numero entero (4 bytes)
.float: Números reales de punto flotante (4 bytes)
.double:Números reales en punto flotante de doble precision (8 bytes)
.char: Caracteres, simbolos alfanumericos (1 byte)
.bool: Representan valores logicos o booleanos (1 byte)

Tipos compuestos: están formados por una composición de tipos simples.

Tabla 2.2 Operaciones con tipos de dato integrados

Tipos de dato integrados	Operaciones
`Números enteros`	`+, -, *, /, %, =, ==, !=, <=, >=vsizeof(), y operaciones con bits` (véase el capítulo 15)
`Números de punto flotante`	`+, -, *, /, =, ==, !=, <=, >=, sizeof()`

Tabla 2.5 Almacenamiento de tipo de datos enteros

Nombre del tipo de datos	Tamaño del **almacenamiento** (en bytes)	Rango de valores
`char`	1	256 caracteres
`bool`	1	verdadero (lo cual es considerado como cualquier valor positivo) y falso (lo cual es un cero)
`short int`	2	–32,768 a +32,767

Tipos de datos con signo y sin signo:
-con signo: negativos y positivos
-sin signo: solo positivos

DETERMINACION DEL TAMAÑO DE ALMACENAMIENTO

Ver donde y como se almacenan los valores:
-sizeof(): proporciona el numero de bytes usados para almacenar valores. Ejemplo
Sizeof (int) → imprime 4

NOTACION EXPLONENCIAL

Los numeros de . flotante pueden escribirse en notación exponencial

Ej: 1625 → 1.652e3
0.00731 → 7.37e-3

CONVERSIONES DE TIPOS Y PRIORIDADES

Expresión: Cualquier combinación de operadores y operandos que pueden ser evaluados para producir un valor.
Automaticos:

a) cuando se asigna un valor de un tipo a la variable de otro tipo
b) Cuando se combinan tipos mixtos en expresiones
c) Cuando se pasan argumentos a funciones

Tabla 2.4 Secuencias de escape

Secuencia de escape	Carácter representado	Significado	Código ASCII
\n	Línea nueva	Se mueve a una línea nueva	00001010
\t	Tabulador horizontal	Se mueve a la siguiente posición del tabulador horizontal	00001001
\v	Tabulador vertical	Se mueve a la siguiente posición del tabulador vertical	00001011
\b	Retroceso	Retrocede un espacio	00001000
\r	Retorno de carro	Mueve el cursor al inicio de la línea actual; se escribe para sobrescribir	00001101
\f	Alimentación de forma	Expulsa una hoja para iniciar otra	00001100
\a	Alerta	Emite una alerta (por lo general un sonido de campana)	00000111
\\	Diagonal inversa	Inserta un carácter de diagonal inversa (ésta se usa para colocar un carácter de diagonal inversa real dentro de una cadena)	01011100
\?	Signo de interrogación	Inserta un carácter de signo de interrogación	00111111
\'	Comilla sencilla	Inserta un carácter de comilla sencilla (ésta se usa para colocar una comilla sencilla interior dentro de un conjunto de comillas sencillas exteriores)	00100111
\"	Comillas dobles	Inserta un carácter de comilla doble (ésta se usa para colocar una comilla doble interior dentro de un conjunto de comillas dobles exteriores)	00100010
nnn	Número octal	El número *nnn* (*n* es un dígito) se considerará un número octal	–
\x*hhhh*	Número hexadecimal	El número *hhhh* (*h* es un dígito) se considerará un número hexadecimal	–
\0	Carácter nulo	Inserta el carácter Null, el cual se define con un valor de 0	00000000

Promoción integral:
Conversion explicita: forza la conversion → (tipo) Valor
Conversión implícita: Tipos básicos mezclados en asignaciones y expresiones, donde los tdd menores se convierten en tdd mayores.

ENTRADA Y SALIDA DESDE ARCHIVOS

Archivo: Información que queda guardada permanentemente. Puede ser de texto (arch de long variable) o binario.
-Pasos para trabajar con archivos: 1) abrir el archivo (var.open(“arch.txt”)) 2)leer o escribir el archivo 3)cerrar el archivo.

-Es necesario incluir la libreria <fstream>
-ES NECESARIO SABER QUE HAY EN EL ARCHIVO Y TIENE QUE RESPETARSE EL ORDEN.

ESTRUCTURAS DE CONTROL: Controlan el flujo de ejecución de las acciones de un programa
-programa con diseño modular.
-Los módulos son diseñados de modo descendente
-Cada modulo se codifica como si fuera un problema diferente.

Existen 3 tipos: 1) Secuencia 2)Selección: If, Else, Switch 3)Repeticion (ver en página 2)
TEOREMA FUNDAMENTAL DE LA PROGRAMACIÓN ESTRUCTURADA:cualquier algoritmo, sin importar cuán complejo fuera, puede ser construido usando combinaciones de tres estructuras de datos estandarizadas:Selección, Iteración y la Secuencia

Declaraciones globales y locales:

1) Variables globales (fuera de una función): Se inicializan con valores nulos
    a) Entidades Globales: se definen fuera del bloque
    b) Entidades locales: se definen dentro del bloque
2) Variables locales: Se inicializan con basura

SELECCIÓN

Sentencia IF
Puede ser completo o incompleto.
No puede tener más de una sentencia si no tiene llaves, y al else se le asigna el IF más cercano que no posea un ELSE.
Sentencia SWITCH (selección múltiple y comportamiento en cascada)
-La primera se utiliza cuando se debe seleccionar una sola de entre múltiples alternativas.
Switch (exp) - case: x {} - break; - Default
El default se ejecuta cuando no se cumple ningún caso
La segunda realiza todo lo que es verdadero, se logra no colocando el break. Del primero que matchea, entran también todos los casos siguientes.

REPETICIÓN - CICLO - ITERACIÓN

- Un bucle es una construccion que repite una sentencia, o una secuencia de sentencias, un múmero de veces. Cada vez que se ejecuta el cuerpo del bucle, es una iteración ejecutada.
- Cuerpo del bucle: Grupo de sentencias que se repiten.

1) **Sentencia FOR (bucle exacto)**

- Se utiliza cuando se sabe cuántas veces se debe repetir el ciclo. for (var control;cond;incremento)

- Orden de ejecución: 1ro) se ejecuta la variable de control, 2do) se ejecuta la condicion y, si ésta es verdadera se ejecuta el cuerpo del bucle (la condición se evalua hasta que sea falsa), 3ro) Si se ejecutó el cuerpo del bucle, se realiza el incremento.
-Anidación de FOR: se utiliza para imprimir muchas filas y columnas.

2) Sentencia WHILE (bucle pretest)

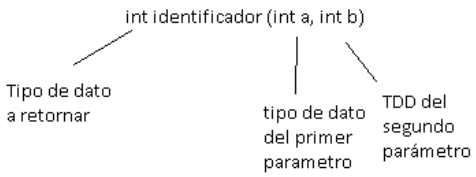
-Permite repetir algo de 0 a "n" veces, pero no se sabe cuántas veces. while (condición)
-La condicion se evalua antes de ejecutar el cuerpo del bucle, y debe tener la variable de control del bucle inicializada antes del ciclo.

a) While controlado por contador: se aumenta un contador hasta hacer falsa la condición
b) While controlado por centinela: se ingresa un valor que termine el bucle. (se ejecuta al menos una vez)

3) Sentencia DO-WHILE (bucle post test)

Permite repetir una secuencia de 1 a "n" veces, es decir, se ejecuta el cuerpo del bucle al menos una vez. Do { } while (cond);

FUNCIONES



"Conjunto de lineas de código que realiza una accion particular", "módulo", "subprograma".

- ➔ Hacen un cálculo o acción particular
- ➔ Recibe uno, 2 o más parámetros
- ➔ Retorna 1, 2, más o ningún valor (void).
- ➔ Pueden ser de tipo bool, int, void,etc.
- ➔ A veces no requieren parámetros (ej, un menú)
- ➔ Una función que no retorna ningún valor se comporta como un procedimiento en otros lenguajes de programación
- ➔ Se utilizan para aplicar la estrategia de diseño descendente Top Down;
  - ◆ Permite la programación modular
  - ◆ Hace el programa más legible
  - ◆ Permite reusar el código
  - ◆ Permite trabajar con librerías (crear librerías propias).
  - ◆ Permite realizar un mejor uso de la memoria. (utiliza la memoria y cuando retorna la destruye)

Función que llama y función llamada → son aquellas funciones que intervienen dentro de otras funciones. La que llama es la que se está ejecutando y la llamada es la que se ejecutará.

Orden de llamada (flujo de ejecución): Siempre que se realice una llamada a una función, la función que se estaba ejecutando previamente queda suspendida, y luego de terminar la ejecución, se destruyen las variables locales creadas y el flujo retorna a la función suspendida anteriormente.
Prototipo de una función: le indica a la función que llama el tipo de valor que será devuelto formalmente, si es que hay alguno, y el tipo de datos y orden de los valores que la función que llama deberá transmitir a la función llamada.
Parámetros de una función:

a) Actuales/Reales: Son aquellos que se utilizan en la invocación de la función (que se encuentran en la función que llama).
b) Formales: Son los que se encuentran en la cabecera, que se utilizan dentro de la función.

Parámetros por copia (int Y, int X): No afectan a las variables de la función, se “copian” los valores que poseen éstas, se asignan a los parámetros formales y cuando la funcion retorna, se eliminan.
Parámetros por referencia (int &Y, int &x): Afectan a las variables de la función. Los parámetros formales reciben la dirección de la variable, y los cambios que se realizan sobre los formales también se realizan sobre las reales.

ARREGLOS / VECTORES

Un arreglo unidimensional, el cual también se conoce como arreglo de dimensión única, es una lista de valores relacionados con el mismo tipo de datos que se almacena usando un nombre de grupo único. *tipo-de-datos nombreArreglo [número-de-elementos]*. ***La buena práctica de programación requiere definir el número de elementos en el arreglo como una constante antes de declarar el arreglo. (puede ser una cte global o una literal)***

Cada elemento en un arreglo se llama elemento o componente del arreglo. Los elementos individuales se almacenan de manera secuencial, con el primer elemento del arreglo almacenado en la primera ubicación reservada, el segundo en la segunda, y así hasta el último. Por esto, Se dice que cada elemento individual es una variable **indexada** o una variable **subindexada**, ya que deben usarse tanto un nombre de variable como un valor índice o subíndice para hacer referencia al elemento. Así, estos elementos se acceden de forma directa (demora el mismo tiempo).

C++ No verifica que los indices esten dentro del rango definido.

Un arreglo posee 3 atributos: La dirección en memoria, el tipo base del arreglo y el tamaño del arreglo.

Tamaño del arreglo: Siempre se determina en tiempo de compilacion (tamaño físico) y se deben almacenar una cantidad de elementos menor o igual al tamaño físico, al que llamaremos TAMAÑO LOGICO

Borrar un elemento de un vector significa desplazar todos los elementos que le siguen una unidad hacia atrás (se reduce el tamaño lógico).

Los arreglos son un tipo de dato **estructurado:** Colección de valores relacionados, se caracterizan por el *Tipo de dato* de sus elementos, la *forma de almacenamiento* y la *forma de acceso*.

Éstos pueden ser:

a) Estáticos: Su tamaño en memoria se mantiene inalterable durante la ejecucion del programa, y ocupan posiciones fijas. (arreglos, cadenas, estructuras)
b) Dinámicos: Su tamaño varía durante el programa y NO ocupan posiciones fijas (listas, pilas, colas, arboles, grafos).

Tipos COMPUESTOS: Se forman combinando tipos simples:

a) Según donde se almacenan: Internos o externos
b) Según los TDD de sus componentes: Homogeneos o no homogeneos
c) Según la implementacion: Provistos por el lenguaje o abstractos (TDA)
d) Según la forma de almacenamiento: Estáticos o Dinámicos.
e)

Hasta acá parcial 1

ARREGLOS MULTIDIMENSIONALES: Funcionan igual que los vectores, sin importar la cantidad de dimensiones. Se almacenan en forma contigua (fila 0, fila 1, fila 2,...,fila n). *Ej: int numeros [filas][columnas].*
- Son arreglos de arreglos
- En las funciones no se especifica la primer dimensión, las siguientes si.
- Son estructuras de datos estáticas y homogéneas (igual que los vectores).

Operaciones con arreglos:
1) Search: Dado en el cuatrimestre anterior
2) Merge
3) Sort

2) Merge: Se aplica cuando se tienen 2 vectores ordenados, se quieren anidar y el vector resultante esté ordenado. Debe aplicarse una estrategia de 3 while (no anidados). El vector final tendrá un tamaño lógico = TL1 + TL2

3) Sort:

- Bubble Sort: Posee complejidad cuadrática, su estrategia es ir llevando el mayor (o menor) al final del vector.
- Bubble Sort Mejorado: Posee la misma complejidad que el anterior, pero éste no realiza las comparaciones innecesarias con los elementos ya ordenados.
- Selección Directa: Complejidad cuadrática, busca el menor y lo intercambia en la posición donde va.
- Inserción Directa: Toma un elemento y lo compara con el anterior, si es menor, lo intercambia, sino, compara con el anterior a ese y así.
- Merge Sort: Es el más eficiente, ya que posee una complejidad de n log (n), divide el vector en subvectores hasta llegar a vectores ordenados (de un solo elemento) y luego les va a aplicando la estrategia del merge.

Algunas definiciones normalmente aplicadas a los algoritmos de ordenamiento:

- Orden definitivo: Es aquel en que cada iteración se pone un elemento en un sitio definitivo (bubble sort, selección directa)
- Estabilidad: Un algoritmo es estable cuando valores iguales permanecen en su lugar relativo (Si no tiene por que intercambiarlos, no debe intercambiarlos).

COMPLEJIDAD: La función de complejidad da que tan rápido crece el tiempo en función de los datos de entrada.

- Eficiencia temporal: Está relacionada con la velocidad de ejecución
    - Factores principales:
        - Técnica utilizada
        - Tamaño de la instancia
    - Otros:
        - Características de la máquina
        - Software
- Eficiencia Espacial: Refiere a la cantidad de recursos utilizados.

La eficiencia depende de las características de los datos, donde hay 3 casos:

- Mejor caso: Minimo tiempo posible (cota inferior)
- Peor caso: Maximo tiempo posible (cota superior)
- Caso medio: Tiempo promedio (promedio entre el maximo y minimo)

Como medir la eficiencia: Se realiza evaluando el peor caso, esta se mide en funcion del tamaño de la entrada o de la función T(n).
T(n) se puede calcular de dos maneras:

- Estimado: Se mide el tiempo de ejecución
- Expresión: Se realiza un analisis matematico del algoritmo. Puede ser exacta o la Expresión Asintótica.

Las operaciones primitivas tienen 1 unidad de tiempo. Todas las T(N) lineales son iguales. En bucles anidados, el total de operaciones es el producto del numero de iteraciones del bucle interno por el bucle externo.
El aumento de la variable determina como T(N) será: ej, i+=2 Lineal, i*=2 Log (n)

CADENAS DE CARACTERES: Es un dato del tipo estructurado, homogéneo y generalmente estáticos.

Diferencias entre:

- Constante de caracter: 'a' no es una cadena, es un valor entero
- Cadena de caracter: Se encuentran entre comillas. Es una serie de caracteres tratados como una unidad. Terminan siempre en un **carácter NULO *0***

Cadenas de caracteres al estilo C: Solo pueden asignarse en la declaración, hay varias funciones que permiten trabajar con ellos en la libreria <cstdio> (como en la declaración lee el espacio como un salto de linea, pueden aplicarse las siguientes funciones) y <cstring> VER TABLA

Strings: Son un tipo de dato dinámico, no tienen longitud máxima, además, son más sofisticadas que las cadenas al estilo C. Cuando se declara una variable string, esta se inicializa vacía.

Ventajas:

- Se pueden concatenar utilizando el operador +
- Se pueden trabajar con subíndices
- Poseen una gran cantidad de métodos que simplifican el código
- Se pueden comparar con operadores relacionales

Desventajas:

- No pueden ser utilizados como campos de una estructura
- Por default pasan por copia

Declaración de un string:

- String cadena1(“hola”);
- String cadena2 = “pepe”;
- String cadena3; // no acumula basura

PUNTEROS: Es un tipo de dato especial, apunta a datos simples o compuestos (Dirección de algo). Es un elemento de bajo nivel.
Permiten simular llamadas por referencia y el manejo dinámico de la memoria.

Int *var; define un puntero var a un tipo de dato entero.

- Se pueden definir punteros a cualquier tipo de dato
- Se inicializan a 0 (NULL, es decir, no apuntan a nada) o a una dirección

  Operador de indirección/desreferencia (*) : Permite desreferenciar el puntero, se obtiene el valor de la variable apuntada.
  Sea int *var; entonces
  - cout<<*var; mostrara por pantalla el valor entero de var, no la direccion en memoria.
  - *var=7; cambiara el valor de la variable apuntada por 7

  * y & son inversos, es decir, &*a=*&a=a

ESTRUCTURAS/REGISTROS: Son TUPLAS de datos, es una coleccion de variables relacionadas bajo un mismo nombre. Pueden tener campos (variables) de diferentes TDD. Se usan para definir registros que van a ser almacenados en archivos o arreglos.
Es una estructura de datos interna, no homogenea, provista por el lenguaje y estaticas.

Definicion de una estructura: realizar esto no ocupa memoria, solo le da informacion al compilador.
Cabe destacar que se pueden utilizar estructuras dentro de otra estructura (registro anidado)

```
struct nombre{

    int documento;
    char apellido[20];

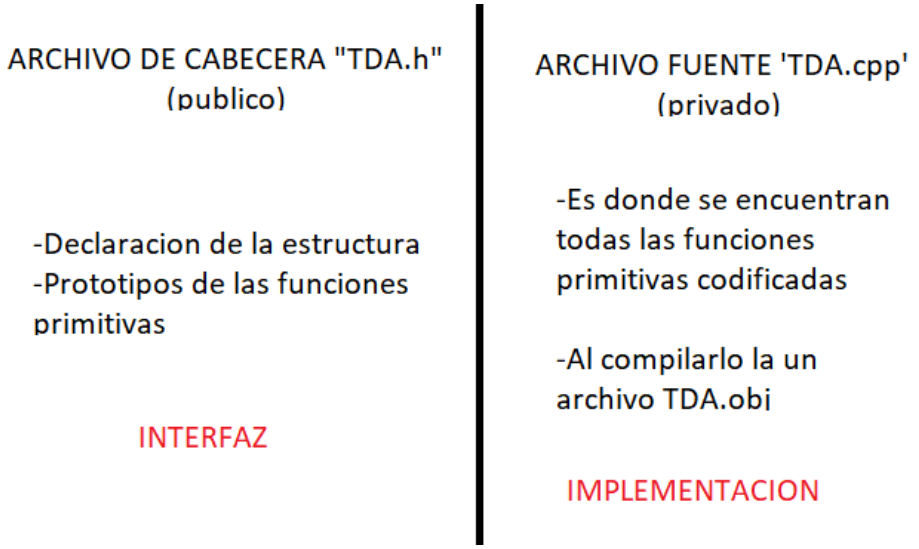
};
```

Ventajas y desventajas entre structs y arreglos:

- Con una sola sentencia en los struct, se pueden copiar todos los campos de una variable a otra (del mismo tipo)
- Los structs pasan por copia por default
- Si el campo es un TDD simple, se puede aplicar cin y cout
- Es posible pasar un unico parametro un campo en una funcion

TIPOS DE DATOS ABSTRACTOS: Es un TDD definido por el programador que se puede manipular de un modo similar a los TDD provistos por el lenguaje. Está formado por un conjunto de valores válidos de datos y un conjunto de operaciones primitivas que se pueden realizar sobre esos valores.

Un TDA se divide en dos partes:
- INTERFAZ: Son las operaciones que son visibles al exterior (los prototipos)
- IMPLEMENTACION: Es donde se realiza la eleccion de las EDD y de los algoritmos



Ventajas:
- Permiten una mejor conceptualización y modelización del mundo real
- Mejora la robustez del sistema
- Mejora el rendimiento
- Separa la Implementación de la Especificación
- Permite la EXTENSIBILIDAD del sistema

ARCHIVOS
Son objetos lógicos que representan un fichero.
- Secuenciales: El orden de acceso está determinado
- Aleatorios/Directo(binarios): Permiten acceder de forma directa a algún elemento del fichero. Están basados en una estructura de registros (todos de igual tamaño).

EDD DINÁMICAS
Son estructuras que varian su tamaño durante la ejecución de un programa: arrays dinamicos, listas enlazadas, pilas, colas, etc.

Listas enlazadas: Son una EDDD, en donde la insercion o borrado de un nodo tiene complejidad constante, pero el acceso al elemento “i” posee complejidad lineal.

- Instrucción **new:** Permite reservar memoria de un numero especifico de bytes, retorna un un puntero void*. En el caso de que no haya memoria, al puntero se le asigna NULL.
- Instrucción **delete:** libera memoria asignada previamente por new, si se le agregan corchetes permite eliminar un vector dinámico completo. Si se deja memoria descolgada, esta quedara bloqueada hasta reiniciar la PC.

```
Struct nodo{
        Int dato;
        Struct nodo * proximo;
};
```